

Verif-Comp

A verifying compiler implementation for a simplified imperative programming language

Contributors

Martin Tesar
Karen Sim
Gurmukh Kharod
Jun Hyeok Park

Language Specification

The following is the grammar of our language. Bolded elements are literal characters. The initial symbol is PROGRAM.

```
PROGRAM → BLOCK
BLOCK → { STATEMENT* }
STATEMENT → ASSIGN | IF | CHECK | PRINT
ASSIGN → <label> := INTEXPR
IF → if BOOLEXP BLOCK else BLOCK
CHECK → check ( BOOLEXP )
PRINT → print ( <label> )

INTEXPR → MULEXP ( ( + | - ) MULEXP )*
MULEXP → MINUSEXP ( * INTEXPR )*
MINUSEXP → PARENEXP | - PARENEXP
PARENEXP → <int literal> | <label> | ( INTEXPR )

BOOLEXP → ANDEXP ( || ANDEXP )*
ANDEXP → NOTEXP ( && NOTEXP )*
NOTEXP → BPARENEXP | ! BPARENEXP
BPARENEXP → INTEXP ( == | > | < ) INTEXP | ( BOOLEXP )

<label> → ([a-z]|[A-Z]) ([a-z]|[A-Z]|[0-9])*
<int literal> → [0-9]+
```

The `havoc` branch has the following modifications to the grammar:

```
...
STATEMENT → ASSIGN | IF | CHECK | PRINT
ASSIGN → <label> := INTEXPR | <label> <<
...
INTEXPR → MINUSEXP ( ( + | - ) MINUSEXP )*
...
```

Any variables must be declared by an assignment expression before they can be used. A variable is considered within the scope of the block if every control path leading up to that point in the block declares the variable. This will be elaborated on in the section on interesting problems to be solved.

Check statements are just assertions on the program state, as discussed in class. The check is verified before the program is interpreted.

The only types of variables and literals are arbitrary precision integers.

This is a sample program that prints x : 0 to the console:

```
{
  x := 36
  y := 42
  if x > y {
    z := x - y
    t := 26
    if z == 1 {
      x := 1
    }
    else {
      x := -1
    }
    check(x == 1 || x == -1)
  }
  else {
    z := y - x
    x := y * z - z * 1 * y + (1 - 2 * 1) * (3 - 2 - 1)
  }
  check( (! z < 0 && x > -2 && x < 2) || 1 == 0 )
  print(x)
}
```

How to use the interpreter

The GitHub repository can be found at <https://github.com/martin-tesar-DGBR/verif-comp>. Clone the project. Choose which tag to check out, v1.1 or v1.0-havoc.

To build the project, in the terminal go to the top level of the project and run `$ mvn package`.

On Linux, run `$ java -jar target/verif-comp-1.0.jar <input>` to run the executable jar.

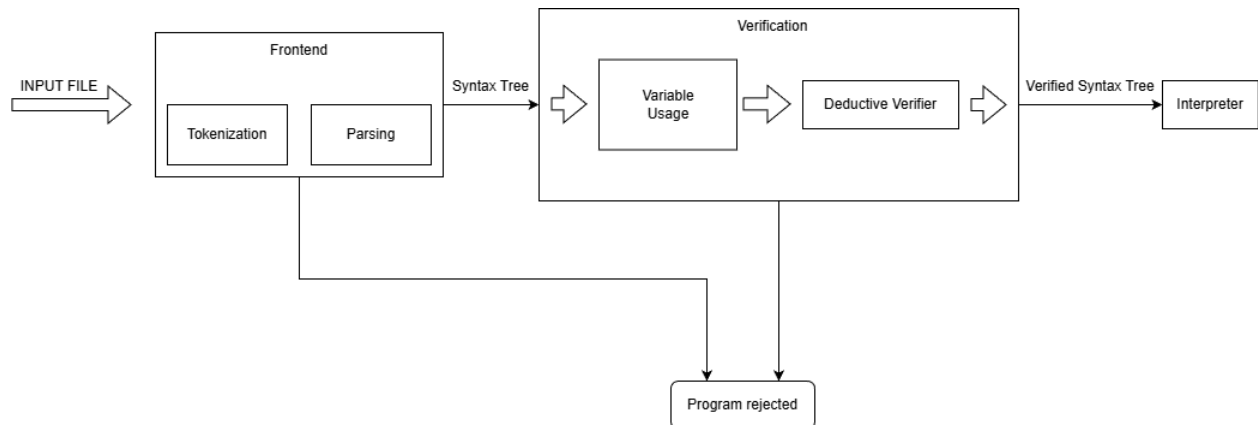
On Windows, run `$ java -jar target\verif-comp-1.0.jar <input>` to run the executable jar.

<input> is the (relative) path to the program to be interpreted.

Note that after building, only the `verif-comp-1.0.jar` file and the `lib/` directory are needed; the JAR file expects the library folder to have the same structure as it has when built in the target directory.

There are two main branches: `master`, which has multiplication at the expense of arbitrary variable inputs, and `havoc`, which has user input at the expense of multiplication. It is advised to run `$ mvn clean` before recompiling the project when switching branches. The decision to have two different versions of the project and their implications will be discussed.

Implementation Overview



Much of the project was implementing a compiler frontend, which includes turning a file into an abstract representation and verifying the program is correct according to the specification. The interpreter is included to demonstrate the runtime effects of a program.

The structure of the project can be seen as a pipeline, where each stage can either pass its output to the next stage or abort if it encounters an error. The part in the diagram labelled Frontend accepts a file and constructs the syntax tree of the program. The verification part of the diagram accepts a syntax tree and passes the same tree along if the program is correct, or rejects it if incorrect according to the specification. Checking the scopes of variables and the entire deductive verifier is part of this stage of the pipeline. A simplification made in the project is that we only allow arbitrary precision integer types for variables, which allows us to skip typechecking altogether. Finally, the interpreter accepts the verified syntax tree and simulates the program's execution.

Example programs can be found in the `src/test/java/verifier/pass/` directory.

Interesting Problems

Deductive Verifier

A program is considered valid iff the Hoare triple $\{\text{true}\}\text{PROGRAM}\{\text{true}\}$ is valid. Theoretically in a deductive verifier like Dafny no assertions need to be made, but the point of these languages is to insert as many correct assertions and requirements as one wishes.

Given the program, we start with the postcondition `true` and wish to construct the weakest precondition of the program, which we can then check the statement $\text{true} \rightarrow \text{wp}(\text{program}, \text{true})$ for validity. This reduces down to checking $\text{wp}(\text{program}, \text{true})$ for validity.

We start by observing that the sequencing rule for the weakest precondition is essentially a backwards traversal of the program's individual statements; as such, we can initialize the weakest precondition to `true` and incrementally update the weakest precondition depending on the statement we encounter as we backtrack through the program. Upon encountering a branch, we duplicate the current weakest precondition, pass it to each branch, and traverse each block separately. When both branches have their weakest precondition computed, we use the weakest precondition rule for if-statements to update the current weakest precondition and continue. As an algorithm, we implemented it in the following way, barring implementation details like the construction of expressions and memory management:

```

WP(Block, Q) :
  for each statement s from end of Block to start of Block:
  
```

```

if s is if-statement:
    then_wp = WP(s.then, Q)
    else_wp = WP(s.else, Q)
    Q = s.condition -> then_wp && s.condition -> else_wp
else if s is check(e):
    Q = e && Q
else if s is assignment x := e:
    Q = Q[e/x]
return Q

```

In the `havoc` branch, there is an additional `else if` for input statements; the relevant pseudocode would be inserted after the assignment check:

```

else if s is input x <<:
    Q =  $\forall y. Q[y/x]$ 

```

Once `wp (PROGRAM, true)` is constructed, we pass the resulting verification condition to Z3 to check for validity.

Variable Scoping Rules

Consider the case when a variable is assigned to within a branch of an if-else statement. If the label is assigned to in both branches then subsequent uses of the variable after the branches are valid. However, if a variable was only declared in one branch and program execution takes the other then subsequent uses of the variable are not well-defined. Clearly there's a criterion for when variables are defined after going through an if-else statement. The following algorithm is what we came up with to determine the set of defined variables in a block:

```

USAGE(Block, vars = {}):
for statement s in Block:
    if s is assignment x := e:
        for variable v that occurs in expression e:
            if v not in vars then return INVALID_USAGE
        vars = vars  $\cup$  {x}
    else if s is if-else statement:
        then_vars = USAGE(s.then, clone(vars))
        else_vars = USAGE(s.else, clone(vars))
        vars = then_vars  $\cap$  else_vars
    else:
        for variable v that occurs in s:
            if v not in vars then return INVALID_USAGE
return vars

```

The idea is that we can introduce a valid variable to the current scope with an assignment, and check usages of variables against the current scope. If a branch occurs, we copy the scope to the blocks of the branch, and the variables defined in both branches will be considered well-defined in the current scope.

The implementation differs slightly in that we don't copy the entire variable scope to each branch, but rather keep a stack of scopes representing the nested blocks. The currently nested scope must check the entire stack for valid usages but can only introduce variables to the innermost scope at the top of the stack. Once a block is finished, it pushes its scope to another stack. When the end of an if-statement is finally reached it intersects the top of the finished stack and pushes it back onto the main one.

In the `havoc` branch, input statements introduce a variable to the current scope, the same as the left hand side of an assignment statement.

User Input

While the compiler correctly verifies every statement in the `master` branch, the apparent application of the verifier seems trivial; one can verify that any assertions hold just by replacing each variable with their respective value and computing the result. Allowing the program to verify statements *without knowing the specific value of a variable ahead of time* would make the applications of deductive verification more apparent.

However, the naive approach of directly adding a *havoc* statement (user input) to the language has a problem: the resulting verification condition comes from the full first order theory of integers, which is known to be undecidable. We could remove multiplication from the language to reduce the verification of a program to deciding the satisfiability of a statement in Presburger arithmetic, which is known to be decidable. This approach has been implemented in the `havoc` branch.

In the `havoc` branch, the interpreter prompts the user to enter a value upon encountering a user input statement. The deductive verifier uses the appropriate *havoc* rule to compute its weakest precondition. Since the weakest precondition rule for *havoc* introduces a universally quantified variable, this makes the application of deductive verification immediately apparent; we can prove statements about the program without knowing the specific value of a variable.

Evaluation

Our test suite covers each stage of the compiler including parsing, interpretation, variable usage, and verification. We tested the correctness of each stage of the compiler pipeline independently using the JUnit testing framework.

To test for correctness, we tested the deductive verifier on 28 programs, including ones with nested if statements, multiple assertions, and assertions in unreachable blocks (assertions in unreachable code are vacuously true so programs with only such assertions should pass). The verifier correctly verifies all valid programs and rejects all invalid ones, i.e. tests with at least one failing assertion.

For testing the runtime of a program, we do warmup runs for each test and then take the average of 30 runs as the runtime. Tests measuring total verification time for programs average 15.78ms, with the majority of time for most tests being spent in Z3 solving the verification condition (VC). VC creation for the test cases averages at about 3.87ms, while Z3 solving takes an average of 10.82ms. For reference, the average number of lines in the test programs is 192, however this is mostly skewed by two test cases with over 1000 lines. Most test cases contain between 10-35 lines.

To evaluate verification performance, we measured the time it took to verify a single program ranging from 1 to 1500 assertions (Table 1). VC generation increases significantly with assertion count, from taking < 2ms for less than 200 assertions to 61ms for 1500 assertions. The average time it took for Z3 to solve VCs across all versions was relatively consistent at 9-12ms, at an average of 11ms. As shown in the table, total time is dominated at first by Z3, and then VC generation as the number of assertions increases.

Table 1

Number of Assertions	Average time for VC generation (ms)	Average time for Z3 to solve VC (ms)	Average total time (ms)
1	0	11	12
10	0	11	12
50	0	11	12
100	1	11	12
200	2	12	14

500	4	12	17
1000	24	9	33
1500	59	9	68

Conclusion

The project is exactly as we envisioned it, barring some niche implementation bug that we haven't seen. An unfortunate result is that the first order theory of integers is undecidable; there was no way to include all three of multiplication, arbitrary assignments, and a decidable theory for the verifier. However, with our architecture it was fairly easy to implement at most two of the choices in separate branches.

From our benchmark, the verifier is reasonably fast on the small programs we have given it. Considering that this is a toy language meant for research purposes, the performance is perfectly adequate for these small problem instances. In this language programs with more assertions than runnable statements are outliers, but it may be worth investigating the simplification of large verification conditions in the future.

Another area worth investigating is in improved debugging capabilities for programs that could not be verified. Currently the verifier builds a single large verification condition for the program, which makes it difficult to find *which* check statement is violated if the verification fails. One approach may be to break up the verification condition into smaller pieces somehow and verify them individually, which would allow for finer debugging capabilities.

Division of Work

Modules

Frontend: Martin
 Variable usage check: Martin
 Deductive verifier: Martin, Karen, Jun
 Interpreter: Gurmukh

Report

Language spec: Martin
 Implementation overview: Martin
 Interesting problems: Martin
 Evaluation: Karen